Undecidability of the Halting Problem

An End-to-End Meta-Math Walkthrough

Dibakar Sigdel[1]

[1] *Mindverse Computing LLC, Lynnwood, WA 98087, USA*

(Dated: October 2026)

Abstract

Assuming a decider allows construction of a machine that reverses its answer on the diagonal input. The contradiction concerns a total decision procedure; direct simulation still provides a one-sided recognition procedure. The report presents the target theorem, declared imports, a dependency graph, five-component node contracts, a continuous proof, a worked example, and a hypothesis-repair audit. It is an authored mathematical exposition, not a deployed-agent transcript or a formal proof-assistant certificate.

LF-05 / Mathematical Logic and Set Theory / Report v1

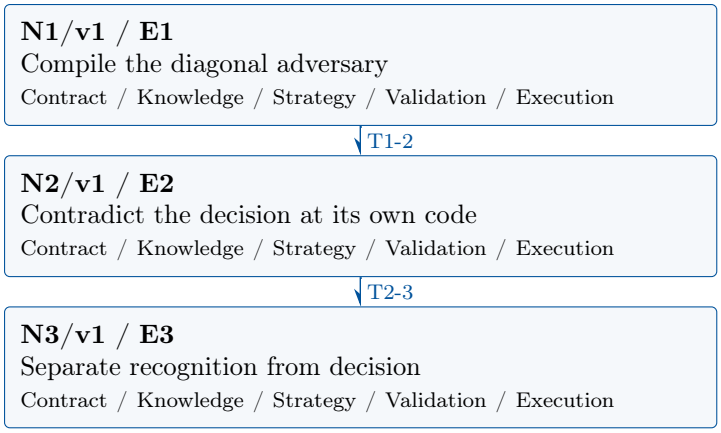
I. TARGET AND IMPORT CONTRACT

Theorem I.1. *Fix an effective encoding of deterministic Turing machines by finite strings. Prove no total computable function $H(e, x) \in \{0, 1\}$ correctly decides for every valid machine code e and input string x whether that machine eventually halts on x. Also prove the halting set is computably enumerable: there is a procedure that halts and accepts exactly the halting pairs. Invalid codes can be assigned a fixed decidable convention and do not affect the result.*

A. Foundations and permitted imports

Use a universal step-by-step simulator for the fixed Turing-machine model, effective composition and conditional branching of finite programs, and the ability to implement an infinite loop. Every such finite program has an encoding in the same model. These are elementary properties of the computational model, not an oracle or a solution of the target decision problem.

II. NODE INTERACTION WORKFLOW



Each box is one mathematical agent containing all five functional components. Blue arrows carry the versioned exports listed below. The layout gives a reading order; only drawn arrows assert dependencies. Diagnostic repair is separate from the proof DAG.

A. Versioned evidence-transfer ledger

T1-2: N1/E1/v1 to N2. There is a machine D with a valid code d such that $D(z)$ loops if $H(z, z) = 1$ and halts if $H(z, z) = 0$.

Receiving activity. N2.1

Compatibility. E1/v1 and the purported correctness of H on the valid pair (d, d). Check: The contradiction is to the existence of a correct total H, not to the existence of the perfectly valid looping machine D.

T2-3: N2/E2/v1 to N3. No total computable halting decider exists.

Receiving activity. N3.2

Compatibility. A valid encoded machine-input pair and the universal simulator; E2/v1 for the distinction from decision. Check: Failure to observe a halt by a finite time does not certify nonhalting.

III. AGENT ROLES AND FIVE-COMPONENT ACTIVITIES

A. N1/v1: Compile the diagonal adversary

Object and role. Construct a finite machine that consults $H(z, z)$ and then deliberately does the opposite. **Local obligation:** There is a machine D with a valid code d such that $D(z)$ loops if $H(z, z) = 1$ and halts if $H(z, z) = 0$.

Inputs. A hypothetical total computable correct decider H.

C — Contract

Publish E1/v1 only after establishing this obligation: There is a machine D with a valid code d such that $D(z)$ loops if $H(z, z) = 1$ and halts if $H(z, z) = 0$.

K — Knowledge

A hypothetical total computable correct decider H. Retain the stated assumptions and import versions.

S — Strategy

Construct a finite machine that consults $H(z, z)$ and then deliberately does the opposite.

V — Validation

Totality makes the branch selection terminate; the constructed machine may itself loop, which is permitted for an ordinary machine.

E — Execution

Carry out activities N1.1–N1.2 below, retaining each calculation and construction as evidence.

Numbered mathematical activities

N1.1 Define a program on input z: compute $H(z, z)$; if the result is one, execute an infinite loop; if the result is zero, halt. Under the hypothesis, the computation of H always finishes and this is an effective finite program.

N1.2 By the fixed model's effective composition and encoding, this program is a Turing machine with some valid code d. Giving a program its own code as an input is legal: codes are finite strings and machines accept finite strings. No machine needs to discover its own code internally.

Export E1/v1. There is a machine D with a valid code d such that $D(z)$ loops if $H(z, z) = 1$ and halts if $H(z, z) = 0$.

Acceptance gate. Totality makes the branch selection terminate; the constructed machine may itself loop, which is permitted for an ordinary machine.

Targeted repair. If the argument assumes a possibly nonterminating test before branching, restore the total-decider hypothesis; without it the reversal would not be implemented on every input. Withhold this export and recheck its descendants after a versioned correction.

B. N2/v1: Contradict the decision at its own code

Object and role. Exhaust both possible Boolean answers and compare them with the constructed behavior. **Local obligation:** No total computable halting decider exists.

Inputs. E1/v1 and the purported correctness of H on the valid pair (d, d). Incoming proof exports: E1/v1 at N2.1.

C — Contract

Publish E2/v1 only after establishing this obligation: No total computable halting decider exists.

K — Knowledge

E1/v1 and the purported correctness of H on the valid pair (d, d). Incoming proof exports: E1/v1 at N2.1. Retain the stated assumptions and import versions.

S — Strategy

Exhaust both possible Boolean answers and compare them with the constructed behavior.

V — Validation

The contradiction is to the existence of a correct total H, not to the existence of the perfectly valid looping machine D.

E — Execution

Carry out activities N2.1–N2.2 below, retaining each calculation and construction as evidence.

Numbered mathematical activities

N2.1 If $H(d, d) = 1$, correctness predicts that $D(d)$ halts. But by construction that branch loops forever, a contradiction.

N2.2 If $H(d, d) = 0$, correctness predicts that $D(d)$ never halts. But the other branch halts immediately after the completed call to H, also a contradiction. These exhaust the two outputs, so the supposed decider cannot exist.

Export E2/v1. No total computable halting decider exists.

Acceptance gate. The contradiction is to the existence of a correct total H, not to the existence of the perfectly valid looping machine D.

Targeted repair. If only one answer branch is analyzed, check the other explicitly; undecidability requires defeating every possible returned answer. Withhold this export and recheck its descendants after a versioned correction.

C. N3/v1: Separate recognition from decision

Object and role. Simulate one step at a time and accept when a halt is observed. **Local obligation:** The halting set is computably enumerable but not decidable.

Inputs. A valid encoded machine-input pair and the universal simulator; E2/v1 for the distinction from decision. Incoming proof exports: E2/v1 at N3.2.

C — Contract

Publish E3/v1 only after establishing this obligation: The halting set is computably enumerable but not decidable.

K — Knowledge

A valid encoded machine-input pair and the universal simulator; E2/v1 for the distinction from decision. Incoming proof exports: E2/v1 at N3.2. Retain the stated assumptions and import versions.

S — Strategy

Simulate one step at a time and accept when a halt is observed.

V — Validation

Failure to observe a halt by a finite time does not certify nonhalting.

E — Execution

Carry out activities N3.1–N3.2 below, retaining each calculation and construction as evidence.

Numbered mathematical activities

N3.1 On input (e, x), simulate the machine coded by e on x one step at a time. If the simulated machine reaches a halting state, accept. If it never halts, the simulator continues forever. Thus the procedure accepts exactly the halting pairs.

N3.2 This is a recognizer, not a decider, because nonhalting inputs need not return an answer. In conjunction with E2, it proves that the halting set is computably enumerable and undecidable.

Export E3/v1. The halting set is computably enumerable but not decidable.

Acceptance gate. Failure to observe a halt by a finite time does not certify nonhalting.

Targeted repair. If simulation is presented as a decision procedure, state its nonterminating behavior on negative instances and keep the recognition/decision distinction. Withhold this export and recheck its descendants after a versioned correction.

IV. CONTINUOUS MATHEMATICAL PROOF

A. N1 — Compile the diagonal adversary

1. **N1.1.** Define a program on input z: compute $H(z, z)$; if the result is one, execute an infinite loop; if the result is zero, halt. Under the hypothesis, the computation of H always finishes and this is an effective finite program.

2. **N1.2.** By the fixed model's effective composition and encoding, this program is a Turing machine with some valid code d. Giving a program its own code as an input is legal: codes are finite strings and machines accept finite strings. No machine needs to discover its own code internally.

B. N2 — Contradict the decision at its own code

1. **N2.1.** If $H(d, d) = 1$, correctness predicts that $D(d)$ halts. But by construction that branch loops forever, a contradiction.

2. **N2.2.** If $H(d, d) = 0$, correctness predicts that $D(d)$ never halts. But the other branch halts immediately after the completed call to H, also a contradiction. These exhaust the two outputs, so the supposed decider cannot exist.

C. N3 — Separate recognition from decision

1. **N3.1.** On input (e, x), simulate the machine coded by e on x one step at a time. If the simulated machine reaches a halting state, accept. If it never halts, the simulator continues forever. Thus the procedure accepts exactly the halting pairs.

2. **N3.2.** This is a recognizer, not a decider, because nonhalting inputs need not return an answer. In conjunction with E2, it proves that the halting set is computably enumerable and undecidable.

□

V. WORKED ORCHESTRATION

A machine that immediately halts on any input is accepted by the simulator after finitely many steps. A machine with a single transition back to the same nonhalting state is simulated indefinitely. If a hypothetical decider labels the diagonal machine $D(d)$ as halting, its prescribed branch is the loop; if it labels it nonhalting, its branch halts. The contradiction is symbolic and does not claim that such a decider was run.

A. Boundary and hypothesis audit

The theorem does not say no individual nonhalting behavior can be proved. The one-state self-loop is easy to certify, and any fixed finite collection of known examples can be classified. What is impossible is one total computable procedure that is correct on every machine-input pair in the stated unbounded model.

VI. ILLUSTRATIVE FAILURE AND REPAIR

The following is an authored diagnostic scenario. Changed assumptions define a new contract; they do not retroactively prove the original statement.

A. Rejected argument, diagnosis, and correction

Rejected N3 rule: after a million simulated steps without halting, return "never halts." A finite program can count to a million plus one and then halt. **Diagnosis:** Validation identifies finite observation being promoted to an unbounded universal claim. **Repair:** let the recognizer keep running on unresolved inputs; it remains a valid semidecision procedure but no longer claims totality. This is an explicit weakening of the rejected algorithmic claim, consistent with the proved impossibility theorem.

B. Publication and reuse

The corrected export retains its explicit assumptions. A hypothetical revision receives a new version, and each consuming activity is rechecked; this scenario is not a recorded runtime event. **Next direction:** Develop many-one reductions to transfer undecidability to other decision problems, recording the computability and correctness of each reduction.

Provenance. This authored proof is complete relative to its declared imports. Reuse each export with its assumptions, proof, and version. The original Markdown remains available for comparison.

Further reading. Andrew Marks, Set Theory notes for sets and ultraproducts; Berkeley, Mathematical Logic for Mathematicians, Part I for first-order semantics; Andrew Marks, Computability Theory notes for effective procedures and undecidability. The proofs below specify their own set-theoretic and semantic assumptions.